INTRODUCTION TO

Automata Theory, Languages, and Computation

3rd Edition

INTRODUCTION TO

Automata Theory, Languages, and Computation

3rd Edition

JOHN E. HOPCROFT
Cornell University

RAJEEV MOTWANI
Stanford University

JEFFREY D. ULLMAN
Stanford University



Boston San Francisco New York
London Toronto Sydney Tokyo Singapore Madrid
Mexico City Munich Paris Cape Town Hong Kong Montreal

Publisher	Greg Tobin
Executive Editor	Michael Hirsch
Acquisitions Editor	Matt Goldstein
Project Editor	Katherine Harutunian
Associate Managing Editor	Jeffrey Holcomb
Cover Designer	Joyce Cosentino Wells
Digital Assets Manager	Marianne Groth
Media Producer	Bethany Tidd
Marketing Manager	Michelle Brown
Marketing Assistant	Dana Lopreato
Senior Author Support/ Technology Specialist	Joe Vetere
Senior Manufacturing Buyer	Carol Melville
Media Manufacturing Buyer	Ginny Michaud

Many of the exercises that appear in this text use the stems of questions from Gradiance Corporation, which retains the copyright to all such questions.
© Gradiance Corp., 2004–2006

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and Addison-Wesley was aware of a trademark claim, the designations have been printed in initial caps or all caps.

Library of Congress Cataloging-in-Publication Data

Hopcroft, John E., 1939-
Introduction to automata theory, languages, and computation / by John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman. -- 3rd ed.
p. cm.
Includes bibliographical references and index.
ISBN 0-321-45536-3
1. Machine theory. 2. Formal languages. 3. Computational complexity. I. Motwani, Rajeev. II. Ullman, Jeffrey D., 1942- III. Title.
QA267.H56 2006
511.3'5--dc22

2006014263

Copyright © 2007 Pearson Education, Inc. All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of the publisher. Printed in the United States of America. For information on obtaining permission for use of material in this work, please submit a written request to Pearson Education, Inc., Rights and Contracts Department, 75 Arlington Street, Suite 300, Boston, MA 02116, fax your request to 617-848-7047, or e-mail at http://www.pearsoned.com/legal/permissions.htm.

1 2 3 4 5 6 7 8 9 10—CW—10 09 08 07 06

Preface

In the preface from the 1979 predecessor to this book, Hopcroft and Ullman marveled at the fact that the subject of automata had exploded, compared with its state at the time they wrote their first book, in 1969. Truly, the 1979 book contained many topics not found in the earlier work and was about twice its size. If you compare this book with the 1979 book, you will find that, like the automobiles of the 1970's, this book is "larger on the outside, but smaller on the inside." That sounds like a retrograde step, but we are happy with the changes for several reasons.

First, in 1979, automata and language theory was still an area of active research. A purpose of that book was to encourage mathematically inclined students to make new contributions to the field. Today, there is little direct research in automata theory (as opposed to its applications), and thus little motivation for us to retain the succinct, highly mathematical tone of the 1979 book.

Second, the role of automata and language theory has changed over the past two decades. In 1979, automata was largely a graduate-level subject, and we imagined our reader was an advanced graduate student, especially those using the later chapters of the book. Today, the subject is a staple of the undergraduate curriculum. As such, the content of the book must assume less in the way of prerequisites from the student, and therefore must provide more of the background and details of arguments than did the earlier book.

A third change in the environment is that Computer Science has grown to an almost unimaginable degree in the past three decades. While in 1979 it was often a challenge to fill up a curriculum with material that we felt would survive the next wave of technology, today very many subdisciplines compete for the limited amount of space in the undergraduate curriculum.

Fourthly, CS has become a more vocational subject, and there is a severe pragmatism among many of its students. We continue to believe that aspects of automata theory are essential tools in a variety of new disciplines, and we believe that the theoretical, mind-expanding exercises embodied in the typical automata course retain their value, no matter how much the student prefers to learn only the most immediately monetizable technology. However, to assure a continued place for the subject on the menu of topics available to the computer science student, we believe it is necessary to emphasize the applications

along with the mathematics. Thus, we have replaced a number of the more abstruse topics in the earlier book with examples of how the ideas are used today. While applications of automata and language theory to compilers are now so well understood that they are normally covered in a compiler course, there are a variety of more recent uses, including model-checking algorithms to verify protocols and document-description languages that are patterned on context-free grammars.

A final explanation for the simultaneous growth and shrinkage of the book is that we were today able to take advantage of the TEX and LATEX typesetting systems developed by Don Knuth and Les Lamport. The latter, especially, encourages the "open" style of typesetting that makes books larger, but easier to read. We appreciate the efforts of both men.

Use of the Book

This book is suitable for a quarter or semester course at the Junior level or above. At Stanford, we have used the notes in CS154, the course in automata and language theory. It is a one-quarter course, which both Rajeev and Jeff have taught. Because of the limited time available, Chapter 11 is not covered, and some of the later material, such as the more difficult polynomial-time reductions in Section 10.4 are omitted as well. The book's Web site (see below) includes notes and syllabi for several offerings of CS154.

Some years ago, we found that many graduate students came to Stanford with a course in automata theory that did not include the theory of intractability. As the Stanford faculty believes that these ideas are essential for every computer scientist to know at more than the level of "NP-complete means it takes too long," there is another course, CS154N, that students may take to cover only Chapters 8, 9, and 10. They actually participate in roughly the last third of CS154 to fulfill the CS154N requirement. Even today, we find several students each quarter availing themselves of this option. Since it requires little extra effort, we recommend the approach.

Prerequisites

To make best use of this book, students should have taken previously a course covering discrete mathematics, e.g., graphs, trees, logic, and proof techniques. We assume also that they have had several courses in programming, and are familiar with common data structures, recursion, and the role of major system components such as compilers. These prerequisites should be obtained in a typical freshman-sophomore CS program.

Exercises

The book contains extensive exercises, with some for almost every section. We indicate harder exercises or parts of exercises with an exclamation point. The hardest exercises have a double exclamation point.

Some of the exercises or parts are marked with a star. For these exercises, we shall endeavor to maintain solutions accessible through the book's Web page. These solutions are publicly available and should be used for self-testing. Note that in a few cases, one exercise B asks for modification or adaptation of your solution to another exercise A. If certain parts of A have solutions, then you should expect the corresponding parts of B to have solutions as well.

Gradiance On-Line Homeworks

A new feature of the third edition is that there is an accompanying set of on-line homeworks using a technology developed by Gradiance Corp. Instructors may assign these homeworks to their class, or students not enrolled in a class may enroll in an "omnibus class" that allows them to do the homeworks as a tutorial (without an instructor-created class). Gradiance questions look like ordinary questions, but your solutions are sampled. If you make an incorrect choice you are given specific advice or feedback to help you correct your solution. If your instructor permits, you are allowed to try again, until you get a perfect score.

A subscription to the Gradiance service is offered with all new copies of this text sold in North America. For more information, visit the Addison-Wesley web site `www.aw.com/gradiance` or send email to `computing@aw.com`.

Support on the World Wide Web

The book's home page is

`http://www-db.stanford.edu/~ullman/ialc.html`

Here are solutions to starred exercises, errata as we learn of them, and backup materials. We hope to make available the notes for each offering of CS154 as we teach it, including homeworks, solutions, and exams.

Acknowledgements

A handout on "how to do proofs" by Craig Silverstein influenced some of the material in Chapter 1. Comments and errata on drafts of the second edition (2000) were received from: Zoe Abrams, George Candea, Haowen Chen, Byong-Gun Chun, Jeffrey Shallit, Bret Taylor, Jason Townsend, and Erik Uzureau.

We also received many emails pointing out errata in the second edition of this book, and these were acknowledged on-line in the errata sheets for that

edition. However, we would like to mention here the following people who provided large numbers of significant errata: Zeki Bayram, Sebastian Hick, Kang-Rae Lee, Christian Lemburg, Nezam Mahdavi-Amiri, Dave Maier, A. P. Marathe, Mark Meuleman, Mustafa Sait-Ametov, Alexey Sarytchev, Jukka Suomela, Rod Topor, Po-Lian Tsai, Tom Whaley, Aaron Windsor, and Jacinth H.T. Wu.

The help of all these people is greatefully acknowledged. Remaining errors are ours, of course.

J. E. H.
R. M.
J. D. U.
Ithaca NY and Stanford CA
February, 2006

Table of Contents

1 Automata: The Methods and the Madness **1**

1.1 Why Study Automata Theory? 2
1.1.1 Introduction to Finite Automata 2
1.1.2 Structural Representations 4
1.1.3 Automata and Complexity 5
1.2 Introduction to Formal Proof 5
1.2.1 Deductive Proofs 6
1.2.2 Reduction to Definitions 8
1.2.3 Other Theorem Forms 10
1.2.4 Theorems That Appear Not to Be If-Then Statements . . 13
1.3 Additional Forms of Proof 13
1.3.1 Proving Equivalences About Sets 14
1.3.2 The Contrapositive 14
1.3.3 Proof by Contradiction 16
1.3.4 Counterexamples 17
1.4 Inductive Proofs 19
1.4.1 Inductions on Integers 19
1.4.2 More General Forms of Integer Inductions 22
1.4.3 Structural Inductions 23
1.4.4 Mutual Inductions 26
1.5 The Central Concepts of Automata Theory 28
1.5.1 Alphabets 28
1.5.2 Strings 29
1.5.3 Languages 30
1.5.4 Problems 31
1.6 Summary of Chapter 1 33
1.7 Gradiance Problems for Chapter 1 35
1.8 References for Chapter 1 36

2 Finite Automata **37**

2.1 An Informal Picture of Finite Automata 38
2.1.1 The Ground Rules 38
2.1.2 The Protocol 39

2.1.3 Enabling the Automata to Ignore Actions 41
2.1.4 The Entire System as an Automaton 43
2.1.5 Using the Product Automaton to Validate the Protocol . 44
2.2 Deterministic Finite Automata 45
2.2.1 Definition of a Deterministic Finite Automaton 45
2.2.2 How a DFA Processes Strings 46
2.2.3 Simpler Notations for DFA's 47
2.2.4 Extending the Transition Function to Strings 49
2.2.5 The Language of a DFA 52
2.2.6 Exercises for Section 2.2 52
2.3 Nondeterministic Finite Automata 55
2.3.1 An Informal View of Nondeterministic Finite Automata . 55
2.3.2 Definition of Nondeterministic Finite Automata 57
2.3.3 The Extended Transition Function 58
2.3.4 The Language of an NFA 59
2.3.5 Equivalence of Deterministic and Nondeterministic Finite Automata 60
2.3.6 A Bad Case for the Subset Construction 64
2.3.7 Exercises for Section 2.3 65
2.4 An Application: Text Search 68
2.4.1 Finding Strings in Text 68
2.4.2 Nondeterministic Finite Automata for Text Search 69
2.4.3 A DFA to Recognize a Set of Keywords 70
2.4.4 Exercises for Section 2.4 71
2.5 Finite Automata With Epsilon-Transitions 72
2.5.1 Uses of ϵ-Transitions 72
2.5.2 The Formal Notation for an ϵ-NFA 73
2.5.3 Epsilon-Closures 74
2.5.4 Extended Transitions and Languages for ϵ-NFA's 75
2.5.5 Eliminating ϵ-Transitions 77
2.5.6 Exercises for Section 2.5 79
2.6 Summary of Chapter 2 80
2.7 Gradiance Problems for Chapter 2 80
2.8 References for Chapter 2 83

3 Regular Expressions and Languages 85
3.1 Regular Expressions 85
3.1.1 The Operators of Regular Expressions 86
3.1.2 Building Regular Expressions 87
3.1.3 Precedence of Regular-Expression Operators 90
3.1.4 Exercises for Section 3.1 91
3.2 Finite Automata and Regular Expressions 92
3.2.1 From DFA's to Regular Expressions 93
3.2.2 Converting DFA's to Regular Expressions by Eliminating States 98

3.2.3 Converting Regular Expressions to Automata 102
3.2.4 Exercises for Section 3.2 107
3.3 Applications of Regular Expressions 109
3.3.1 Regular Expressions in UNIX 109
3.3.2 Lexical Analysis 110
3.3.3 Finding Patterns in Text 112
3.3.4 Exercises for Section 3.3 114
3.4 Algebraic Laws for Regular Expressions 115
3.4.1 Associativity and Commutativity 115
3.4.2 Identities and Annihilators 116
3.4.3 Distributive Laws 116
3.4.4 The Idempotent Law 117
3.4.5 Laws Involving Closures 118
3.4.6 Discovering Laws for Regular Expressions 118
3.4.7 The Test for a Regular-Expression Algebraic Law 120
3.4.8 Exercises for Section 3.4 121
3.5 Summary of Chapter 3 123
3.6 Gradiance Problems for Chapter 3 123
3.7 References for Chapter 3 125

4 Properties of Regular Languages 127
4.1 Proving Languages Not to Be Regular 128
4.1.1 The Pumping Lemma for Regular Languages 128
4.1.2 Applications of the Pumping Lemma 129
4.1.3 Exercises for Section 4.1 131
4.2 Closure Properties of Regular Languages 133
4.2.1 Closure of Regular Languages Under Boolean Operations 133
4.2.2 Reversal 139
4.2.3 Homomorphisms 140
4.2.4 Inverse Homomorphisms 142
4.2.5 Exercises for Section 4.2 147
4.3 Decision Properties of Regular Languages 150
4.3.1 Converting Among Representations 151
4.3.2 Testing Emptiness of Regular Languages 153
4.3.3 Testing Membership in a Regular Language 154
4.3.4 Exercises for Section 4.3 155
4.4 Equivalence and Minimization of Automata 155
4.4.1 Testing Equivalence of States 155
4.4.2 Testing Equivalence of Regular Languages 159
4.4.3 Minimization of DFA's 160
4.4.4 Why the Minimized DFA Can't Be Beaten 163
4.4.5 Exercises for Section 4.4 165
4.5 Summary of Chapter 4 166
4.6 Gradiance Problems for Chapter 4 167
4.7 References for Chapter 4 169

5 Context-Free Grammars and Languages 171
5.1 Context-Free Grammars . . . 171
5.1.1 An Informal Example . . . 172
5.1.2 Definition of Context-Free Grammars . . . 173
5.1.3 Derivations Using a Grammar . . . 175
5.1.4 Leftmost and Rightmost Derivations . . . 177
5.1.5 The Language of a Grammar . . . 179
5.1.6 Sentential Forms . . . 180
5.1.7 Exercises for Section 5.1 . . . 181
5.2 Parse Trees . . . 183
5.2.1 Constructing Parse Trees . . . 183
5.2.2 The Yield of a Parse Tree . . . 185
5.2.3 Inference, Derivations, and Parse Trees . . . 185
5.2.4 From Inferences to Trees . . . 187
5.2.5 From Trees to Derivations . . . 188
5.2.6 From Derivations to Recursive Inferences . . . 191
5.2.7 Exercises for Section 5.2 . . . 193
5.3 Applications of Context-Free Grammars . . . 193
5.3.1 Parsers . . . 194
5.3.2 The YACC Parser-Generator . . . 196
5.3.3 Markup Languages . . . 197
5.3.4 XML and Document-Type Definitions . . . 200
5.3.5 Exercises for Section 5.3 . . . 206
5.4 Ambiguity in Grammars and Languages . . . 207
5.4.1 Ambiguous Grammars . . . 207
5.4.2 Removing Ambiguity From Grammars . . . 209
5.4.3 Leftmost Derivations as a Way to Express Ambiguity . . . 212
5.4.4 Inherent Ambiguity . . . 213
5.4.5 Exercises for Section 5.4 . . . 215
5.5 Summary of Chapter 5 . . . 216
5.6 Gradiance Problems for Chapter 5 . . . 218
5.7 References for Chapter 5 . . . 224

6 Pushdown Automata 225
6.1 Definition of the Pushdown Automaton . . . 225
6.1.1 Informal Introduction . . . 225
6.1.2 The Formal Definition of Pushdown Automata . . . 227
6.1.3 A Graphical Notation for PDA's . . . 229
6.1.4 Instantaneous Descriptions of a PDA . . . 230
6.1.5 Exercises for Section 6.1 . . . 233
6.2 The Languages of a PDA . . . 234
6.2.1 Acceptance by Final State . . . 235
6.2.2 Acceptance by Empty Stack . . . 236
6.2.3 From Empty Stack to Final State . . . 237
6.2.4 From Final State to Empty Stack . . . 240

6.2.5 Exercises for Section 6.2 . . . 241
6.3 Equivalence of PDA's and CFG's . . . 243
6.3.1 From Grammars to Pushdown Automata . . . 243
6.3.2 From PDA's to Grammars . . . 247
6.3.3 Exercises for Section 6.3 . . . 251
6.4 Deterministic Pushdown Automata . . . 252
6.4.1 Definition of a Deterministic PDA . . . 252
6.4.2 Regular Languages and Deterministic PDA's . . . 253
6.4.3 DPDA's and Context-Free Languages . . . 254
6.4.4 DPDA's and Ambiguous Grammars . . . 255
6.4.5 Exercises for Section 6.4 . . . 256
6.5 Summary of Chapter 6 . . . 257
6.6 Gradiance Problems for Chapter 6 . . . 258
6.7 References for Chapter 6 . . . 260

7 Properties of Context-Free Languages 261
7.1 Normal Forms for Context-Free Grammars . . . 261
7.1.1 Eliminating Useless Symbols . . . 262
7.1.2 Computing the Generating and Reachable Symbols . . . 264
7.1.3 Eliminating ϵ-Productions . . . 265
7.1.4 Eliminating Unit Productions . . . 268
7.1.5 Chomsky Normal Form . . . 272
7.1.6 Exercises for Section 7.1 . . . 275
7.2 The Pumping Lemma for Context-Free Languages . . . 279
7.2.1 The Size of Parse Trees . . . 280
7.2.2 Statement of the Pumping Lemma . . . 280
7.2.3 Applications of the Pumping Lemma for CFL's . . . 283
7.2.4 Exercises for Section 7.2 . . . 286
7.3 Closure Properties of Context-Free Languages . . . 287
7.3.1 Substitutions . . . 287
7.3.2 Applications of the Substitution Theorem . . . 289
7.3.3 Reversal . . . 290
7.3.4 Intersection With a Regular Language . . . 291
7.3.5 Inverse Homomorphism . . . 295
7.3.6 Exercises for Section 7.3 . . . 297
7.4 Decision Properties of CFL's . . . 299
7.4.1 Complexity of Converting Among CFG's and PDA's . . . 299
7.4.2 Running Time of Conversion to Chomsky Normal Form . . . 301
7.4.3 Testing Emptiness of CFL's . . . 302
7.4.4 Testing Membership in a CFL . . . 303
7.4.5 Preview of Undecidable CFL Problems . . . 307
7.4.6 Exercises for Section 7.4 . . . 307
7.5 Summary of Chapter 7 . . . 308
7.6 Gradiance Problems for Chapter 7 . . . 309
7.7 References for Chapter 7 . . . 314

8 Introduction to Turing Machines **315**
8.1 Problems That Computers Cannot Solve . . . 315
8.1.1 Programs that Print "Hello, World" . . . 316
8.1.2 The Hypothetical "Hello, World" Tester . . . 318
8.1.3 Reducing One Problem to Another . . . 321
8.1.4 Exercises for Section 8.1 . . . 324
8.2 The Turing Machine . . . 324
8.2.1 The Quest to Decide All Mathematical Questions . . . 325
8.2.2 Notation for the Turing Machine . . . 326
8.2.3 Instantaneous Descriptions for Turing Machines . . . 327
8.2.4 Transition Diagrams for Turing Machines . . . 331
8.2.5 The Language of a Turing Machine . . . 334
8.2.6 Turing Machines and Halting . . . 334
8.2.7 Exercises for Section 8.2 . . . 335
8.3 Programming Techniques for Turing Machines . . . 337
8.3.1 Storage in the State . . . 337
8.3.2 Multiple Tracks . . . 339
8.3.3 Subroutines . . . 341
8.3.4 Exercises for Section 8.3 . . . 343
8.4 Extensions to the Basic Turing Machine . . . 343
8.4.1 Multitape Turing Machines . . . 344
8.4.2 Equivalence of One-Tape and Multitape TM's . . . 345
8.4.3 Running Time and the Many-Tapes-to-One Construction 346
8.4.4 Nondeterministic Turing Machines . . . 347
8.4.5 Exercises for Section 8.4 . . . 349
8.5 Restricted Turing Machines . . . 352
8.5.1 Turing Machines With Semi-infinite Tapes . . . 352
8.5.2 Multistack Machines . . . 355
8.5.3 Counter Machines . . . 358
8.5.4 The Power of Counter Machines . . . 359
8.5.5 Exercises for Section 8.5 . . . 361
8.6 Turing Machines and Computers . . . 362
8.6.1 Simulating a Turing Machine by Computer . . . 362
8.6.2 Simulating a Computer by a Turing Machine . . . 363
8.6.3 Comparing the Running Times of Computers and Turing Machines . . . 368
8.7 Summary of Chapter 8 . . . 370
8.8 Gradiance Problems for Chapter 8 . . . 372
8.9 References for Chapter 8 . . . 374

9 Undecidability **377**
9.1 A Language That Is Not Recursively Enumerable . . . 378
9.1.1 Enumerating the Binary Strings . . . 379
9.1.2 Codes for Turing Machines . . . 379
9.1.3 The Diagonalization Language . . . 380

9.1.4 Proof That L_d Is Not Recursively Enumerable 382
9.1.5 Exercises for Section 9.1 382
9.2 An Undecidable Problem That Is RE 383
9.2.1 Recursive Languages 383
9.2.2 Complements of Recursive and RE languages 384
9.2.3 The Universal Language 387
9.2.4 Undecidability of the Universal Language 389
9.2.5 Exercises for Section 9.2 390
9.3 Undecidable Problems About Turing Machines 392
9.3.1 Reductions 392
9.3.2 Turing Machines That Accept the Empty Language . . . 394
9.3.3 Rice's Theorem and Properties of the RE Languages . . . 397
9.3.4 Problems about Turing-Machine Specifications 399
9.3.5 Exercises for Section 9.3 400
9.4 Post's Correspondence Problem 401
9.4.1 Definition of Post's Correspondence Problem 401
9.4.2 The "Modified" PCP 404
9.4.3 Completion of the Proof of PCP Undecidability 407
9.4.4 Exercises for Section 9.4 412
9.5 Other Undecidable Problems 412
9.5.1 Problems About Programs 413
9.5.2 Undecidability of Ambiguity for CFG's 413
9.5.3 The Complement of a List Language 415
9.5.4 Exercises for Section 9.5 418
9.6 Summary of Chapter 9 419
9.7 Gradiance Problems for Chapter 9 420
9.8 References for Chapter 9 422

10 Intractable Problems **425**
10.1 The Classes $\mathcal{P}$ and $\mathcal{NP}$ 426
10.1.1 Problems Solvable in Polynomial Time 426
10.1.2 An Example: Kruskal's Algorithm 426
10.1.3 Nondeterministic Polynomial Time 431
10.1.4 An $\mathcal{NP}$ Example: The Traveling Salesman Problem . . . 431
10.1.5 Polynomial-Time Reductions 433
10.1.6 NP-Complete Problems 434
10.1.7 Exercises for Section 10.1 435
10.2 An NP-Complete Problem 438
10.2.1 The Satisfiability Problem 438
10.2.2 Representing SAT Instances 439
10.2.3 NP-Completeness of the SAT Problem 440
10.2.4 Exercises for Section 10.2 447
10.3 A Restricted Satisfiability Problem 447
10.3.1 Normal Forms for Boolean Expressions 448
10.3.2 Converting Expressions to CNF 449

10.3.3 NP-Completeness of CSAT . . . 452
10.3.4 NP-Completeness of 3SAT . . . 456
10.3.5 Exercises for Section 10.3 . . . 458
10.4 Additional NP-Complete Problems . . . 458
10.4.1 Describing NP-complete Problems . . . 459
10.4.2 The Problem of Independent Sets . . . 459
10.4.3 The Node-Cover Problem . . . 463
10.4.4 The Directed Hamilton-Circuit Problem . . . 465
10.4.5 Undirected Hamilton Circuits and the TSP . . . 471
10.4.6 Summary of NP-Complete Problems . . . 473
10.4.7 Exercises for Section 10.4 . . . 473
10.5 Summary of Chapter 10 . . . 477
10.6 Gradiance Problems for Chapter 10 . . . 478
10.7 References for Chapter 10 . . . 481

11 Additional Classes of Problems **483**
11.1 Complements of Languages in $\mathcal{NP}$ . . . 484
11.1.1 The Class Co-$\mathcal{NP}$ . . . 484
11.1.2 NP-Complete Problems and Co-$\mathcal{NP}$ . . . 485
11.1.3 Exercises for Section 11.1 . . . 486
11.2 Problems Solvable in Polynomial Space . . . 487
11.2.1 Polynomial-Space Turing Machines . . . 487
11.2.2 Relationship of $\mathcal{PS}$ and $\mathcal{NPS}$ to Previously Defined Classes 488
11.2.3 Deterministic and Nondeterministic Polynomial Space . . . 490
11.3 A Problem That Is Complete for $\mathcal{PS}$ . . . 492
11.3.1 PS-Completeness . . . 492
11.3.2 Quantified Boolean Formulas . . . 493
11.3.3 Evaluating Quantified Boolean Formulas . . . 494
11.3.4 PS-Completeness of the QBF Problem . . . 496
11.3.5 Exercises for Section 11.3 . . . 501
11.4 Language Classes Based on Randomization . . . 501
11.4.1 Quicksort: an Example of a Randomized Algorithm . . . 502
11.4.2 A Turing-Machine Model Using Randomization . . . 503
11.4.3 The Language of a Randomized Turing Machine . . . 504
11.4.4 The Class $\mathcal{RP}$ . . . 506
11.4.5 Recognizing Languages in $\mathcal{RP}$ . . . 508
11.4.6 The Class $\mathcal{ZPP}$ . . . 509
11.4.7 Relationship Between $\mathcal{RP}$ and $\mathcal{ZPP}$ . . . 510
11.4.8 Relationships to the Classes $\mathcal{P}$ and $\mathcal{NP}$ . . . 511
11.5 The Complexity of Primality Testing . . . 512
11.5.1 The Importance of Testing Primality . . . 512
11.5.2 Introduction to Modular Arithmetic . . . 514
11.5.3 The Complexity of Modular-Arithmetic Computations . . . 516
11.5.4 Random-Polynomial Primality Testing . . . 517
11.5.5 Nondeterministic Primality Tests . . . 518

11.5.6 Exercises for Section 11.5 521
11.6 Summary of Chapter 11 522
11.7 Gradiance Problems for Chapter 11 523
11.8 References for Chapter 11 524

Index **527**

Chapter 1

Automata: The Methods and the Madness

Automata theory is the study of abstract computing devices, or "machines." Before there were computers, in the 1930's, A. Turing studied an abstract machine that had all the capabilities of today's computers, at least as far as in what they could compute. Turing's goal was to describe precisely the boundary between what a computing machine could do and what it could not do; his conclusions apply not only to his abstract *Turing machines*, but to today's real machines.

In the 1940's and 1950's, simpler kinds of machines, which we today call "finite automata," were studied by a number of researchers. These automata, originally proposed to model brain function, turned out to be extremely useful for a variety of other purposes, which we shall mention in Section 1.1. Also in the late 1950's, the linguist N. Chomsky began the study of formal "grammars." While not strictly machines, these grammars have close relationships to abstract automata and serve today as the basis of some important software components, including parts of compilers.

In 1969, S. Cook extended Turing's study of what could and what could not be computed. Cook was able to separate those problems that can be solved efficiently by computer from those problems that can in principle be solved, but in practice take so much time that computers are useless for all but very small instances of the problem. The latter class of problems is called "intractable," or "NP-hard." It is highly unlikely that even the exponential improvement in computing speed that computer hardware has been following ("Moore's Law") will have significant impact on our ability to solve large instances of intractable problems.

All of these theoretical developments bear directly on what computer scientists do today. Some of the concepts, like finite automata and certain kinds of formal grammars, are used in the design and construction of important kinds of software. Other concepts, like the Turing machine, help us understand what

we can expect from our software. Especially, the theory of intractable problems lets us deduce whether we are likely to be able to meet a problem "head-on" and write a program to solve it (because it is not in the intractable class), or whether we have to find some way to work around the intractable problem: find an approximation, use a heuristic, or use some other method to limit the amount of time the program will spend solving the problem.

In this introductory chapter, we begin with a very high-level view of what automata theory is about, and what its uses are. Much of the chapter is devoted to a survey of proof techniques and tricks for discovering proofs. We cover deductive proofs, reformulating statements, proofs by contradiction, proofs by induction, and other important concepts. A final section introduces the concepts that pervade automata theory: alphabets, strings, and languages.

1.1 Why Study Automata Theory?

There are several reasons why the study of automata and complexity is an important part of the core of Computer Science. This section serves to introduce the reader to the principal motivation and also outlines the major topics covered in this book.

1.1.1 Introduction to Finite Automata

Finite automata are a useful model for many important kinds of hardware and software. We shall see, starting in Chapter 2, examples of how the concepts are used. For the moment, let us just list some of the most important kinds:

1. Software for designing and checking the behavior of digital circuits.
2. The "lexical analyzer" of a typical compiler, that is, the compiler component that breaks the input text into logical units, such as identifiers, keywords, and punctuation.
3. Software for scanning large bodies of text, such as collections of Web pages, to find occurrences of words, phrases, or other patterns.
4. Software for verifying systems of all types that have a finite number of distinct states, such as communications protocols or protocols for secure exchange of information.

While we shall soon meet a precise definition of automata of various types, let us begin our informal introduction with a sketch of what a finite automaton is and does. There are many systems or components, such as those enumerated above, that may be viewed as being at all times in one of a finite number of "states." The purpose of a state is to remember the relevant portion of the system's history. Since there are only a finite number of states, the entire history generally cannot be remembered, so the system must be designed carefully, to

remember what is important and forget what is not. The advantage of having only a finite number of states is that we can implement the system with a fixed set of resources. For example, we could implement it in hardware as a circuit, or as a simple form of program that can make decisions looking only at a limited amount of data or using the position in the code itself to make the decision.

Example 1.1 : Perhaps the simplest nontrivial finite automaton is an on/off switch. The device remembers whether it is in the "on" state or the "off" state, and it allows the user to press a button whose effect is different, depending on the state of the switch. That is, if the switch is in the off state, then pressing the button changes it to the on state, and if the switch is in the on state, then pressing the same button turns it to the off state.

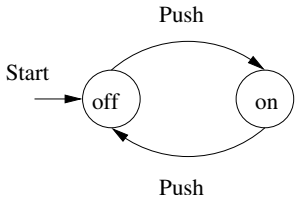


Figure 1.1: A finite automaton modeling an on/off switch

The finite-automaton model for the switch is shown in Fig. 1.1. As for all finite automata, the states are represented by circles; in this example, we have named the states *on* and *off*. Arcs between states are labeled by "inputs," which represent external influences on the system. Here, both arcs are labeled by the input *Push*, which represents a user pushing the button. The intent of the two arcs is that whichever state the system is in, when the *Push* input is received it goes to the other state.

One of the states is designated the "start state," the state in which the system is placed initially. In our example, the start state is *off*, and we conventionally indicate the start state by the word *Start* and an arrow leading to that state.

It is often necessary to indicate one or more states as "final" or "accepting" states. Entering one of these states after a sequence of inputs indicates that the input sequence is good in some way. For instance, we could have regarded the state *on* in Fig. 1.1 as accepting, because in that state, the device being controlled by the switch will operate. It is conventional to designate accepting states by a double circle, although we have not made any such designation in Fig. 1.1. □

Example 1.2 : Sometimes, what is remembered by a state can be much more complex than an on/off choice. Figure 1.2 shows another finite automaton that could be part of a lexical analyzer. The job of this automaton is to recognize the keyword `then`. It thus needs five states, each of which represents a different

position in the word `then` that has been reached so far. These positions correspond to the prefixes of the word, ranging from the empty string (i.e., nothing of the word has been seen so far) to the complete word.

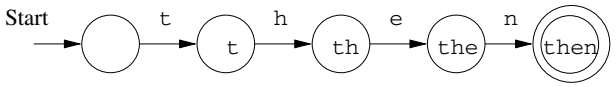


Figure 1.2: A finite automaton modeling recognition of `then`

In Fig. 1.2, the five states are named by the prefix of `then` seen so far. Inputs correspond to letters. We may imagine that the lexical analyzer examines one character of the program that it is compiling at a time, and the next character to be examined is the input to the automaton. The start state corresponds to the empty string, and each state has a transition on the next letter of `then` to the state that corresponds to the next-larger prefix. The state named `then` is entered when the input has spelled the word `then`. Since it is the job of this automaton to recognize when `then` has been seen, we could consider that state the lone accepting state. □

1.1.2 Structural Representations

There are two important notations that are not automaton-like, but play an important role in the study of automata and their applications.

1. *Grammars* are useful models when designing software that processes data with a recursive structure. The best-known example is a "parser," the component of a compiler that deals with the recursively nested features of the typical programming language, such as expressions — arithmetic, conditional, and so on. For instance, a grammatical rule like $E \Rightarrow E + E$ states that an expression can be formed by taking any two expressions and connecting them by a plus sign; this rule is typical of how expressions of real programming languages are formed. We introduce context-free grammars, as they are usually called, in Chapter 5.

2. *Regular Expressions* also denote the structure of data, especially text strings. As we shall see in Chapter 3, the patterns of strings they describe are exactly the same as what can be described by finite automata. The style of these expressions differs significantly from that of grammars, and we shall content ourselves with a simple example here. The UNIX-style regular expression `'[A-Z][a-z]*[ ][A-Z][A-Z]'` represents capitalized words followed by a space and two capital letters. This expression represents patterns in text that could be a city and state, e.g., `Ithaca NY`. It misses multiword city names, such as `Palo Alto CA`, which could be captured by the more complex expression

```
'[A-Z][a-z]*([ ][A-Z][a-z]*)*[ ][A-Z][A-Z]'
```

When interpreting such expressions, we only need to know that `[A-Z]` represents a range of characters from capital "A" to capital "Z" (i.e., any capital letter), and `[ ]` is used to represent the blank character alone. Also, the symbol * represents "any number of" the preceding expression. Parentheses are used to group components of the expression; they do not represent characters of the text described.

1.1.3 Automata and Complexity

Automata are essential for the study of the limits of computation. As we mentioned in the introduction to the chapter, there are two important issues:

1. What can a computer do at all? This study is called "decidability," and the problems that can be solved by computer are called "decidable." This topic is addressed in Chapter 9.

2. What can a computer do efficiently? This study is called "intractability," and the problems that can be solved by a computer using no more time than some slowly growing function of the size of the input are called "tractable." Often, we take all polynomial functions to be "slowly growing," while functions that grow faster than any polynomial are deemed to grow too fast. The subject is studied in Chapter 10.

1.2 Introduction to Formal Proof

If you studied plane geometry in high school any time before the 1990's, you most likely had to do some detailed "deductive proofs," where you showed the truth of a statement by a detailed sequence of steps and reasons. While geometry has its practical side (e.g., you need to know the rule for computing the area of a rectangle if you need to buy the correct amount of carpet for a room), the study of formal proof methodologies was at least as important a reason for covering this branch of mathematics in high school.

In the USA of the 1990's it became popular to teach proof as a matter of personal feelings about the statement. While it is good to feel the truth of a statement you need to use, important techniques of proof are no longer mastered in high school. Yet proof is something that every computer scientist needs to understand. Some computer scientists take the extreme view that a formal proof of the correctness of a program should go hand-in-hand with the writing of the program itself. We doubt that doing so is productive. On the other hand, there are those who say that proof has no place in the discipline of programming. The slogan "if you are not sure your program is correct, run it and see" is commonly offered by this camp.

Our position is between these two extremes. Testing programs is surely essential. However, testing goes only so far, since you cannot try your program on every input. More importantly, if your program is complex — say a tricky

recursion or iteration — then if you don't understand what is going on as you go around a loop or call a function recursively, it is unlikely that you will write the code correctly. When your testing tells you the code is incorrect, you still need to get it right.

To make your iteration or recursion correct, you need to set up an inductive hypothesis, and it is helpful to reason, formally or informally, that the hypothesis is consistent with the iteration or recursion. This process of understanding the workings of a correct program is essentially the same as the process of proving theorems by induction. Thus, in addition to giving you models that are useful for certain types of software, it has become traditional for a course on automata theory to cover methodologies of formal proof. Perhaps more than other core subjects of computer science, automata theory lends itself to natural and interesting proofs, both of the *deductive* kind (a sequence of justified steps) and the *inductive* kind (recursive proofs of a parameterized statement that use the statement itself with "lower" values of the parameter).

1.2.1 Deductive Proofs

As mentioned above, a deductive proof consists of a sequence of statements whose truth leads us from some initial statement, called the *hypothesis* or the *given statement(s)*, to a *conclusion* statement. Each step in the proof must follow, by some accepted logical principle, from either the given facts, or some of the previous statements in the deductive proof, or a combination of these.

The hypothesis may be true or false, typically depending on values of its parameters. Often, the hypothesis consists of several independent statements connected by a logical AND. In those cases, we talk of each of these statements as a hypothesis, or as a given statement.

The theorem that is proved when we go from a hypothesis H to a conclusion C is the statement "if H then C." We say that C is *deduced* from H. An example theorem of the form "if H then C" will illustrate these points.

Theorem 1.3: If $x \geq 4$, then $2^x \geq x^2$. $\square$

It is not hard to convince ourselves informally that Theorem 1.3 is true, although a formal proof requires induction and will be left for Example 1.17. First, notice that the hypothesis H is "$x \geq 4$." This hypothesis has a parameter, x, and thus is neither true nor false. Rather, its truth depends on the value of the parameter x; e.g., H is true for $x = 6$ and false for $x = 2$.

Likewise, the conclusion C is "$2^x \geq x^2$." This statement also uses parameter x and is true for certain values of x and not others. For example, C is false for $x = 3$, since $2^3 = 8$, which is not as large as $3^2 = 9$. On the other hand, C is true for $x = 4$, since $2^4 = 4^2 = 16$. For $x = 5$, the statement is also true, since $2^5 = 32$ is at least as large as $5^2 = 25$.

Perhaps you can see the intuitive argument that tells us the conclusion $2^x \geq x^2$ will be true whenever $x \geq 4$. We already saw that it is true for $x = 4$. As x grows larger than 4, the left side, 2^x doubles each time x increases by

1. However, the right side, x^2, grows by the ratio $\left(\frac{x+1}{x}\right)^2$. If $x \geq 4$, then $(x+1)/x$ cannot be greater than 1.25, and therefore $\left(\frac{x+1}{x}\right)^2$ cannot be bigger than 1.5625. Since $1.5625 < 2$, each time x increases above 4 the left side 2^x grows more than the right side x^2. Thus, as long as we start from a value like $x = 4$ where the inequality $2^x \geq x^2$ is already satisfied, we can increase x as much as we like, and the inequality will still be satisfied.

We have now completed an informal but accurate proof of Theorem 1.3. We shall return to the proof and make it more precise in Example 1.17, after we introduce "inductive" proofs.

Theorem 1.3, like all interesting theorems, involves an infinite number of related facts, in this case the statement "if $x \geq 4$ then $2^x \geq x^2$" for all integers x. In fact, we do not need to assume x is an integer, but the proof talked about repeatedly increasing x by 1, starting at $x = 4$, so we really addressed only the situation where x is an integer.

Theorem 1.3 can be used to help deduce other theorems. In the next example, we consider a complete deductive proof of a simple theorem that uses Theorem 1.3.

Theorem 1.4: If x is the sum of the squares of four positive integers, then $2^x \geq x^2$.

PROOF: The intuitive idea of the proof is that if the hypothesis is true for x, that is, x is the sum of the squares of four positive integers, then x must be at least 4. Therefore, the hypothesis of Theorem 1.3 holds, and since we believe that theorem, we may state that its conclusion is also true for x. The reasoning can be expressed as a sequence of steps. Each step is either the hypothesis of the theorem to be proved, part of that hypothesis, or a statement that follows from one or more previous statements.

By "follows" we mean that if the hypothesis of some theorem is a previous statement, then the conclusion of that theorem is true, and can be written down as a statement of our proof. This logical rule is often called *modus ponens*; i.e., if we know H is true, and we know "if H then C" is true, we may conclude that C is true. We also allow certain other logical steps to be used in creating a statement that follows from one or more previous statements. For instance, if A and B are two previous statements, then we can deduce and write down the statement "A and B."

Figure 1.3 shows the sequence of statements we need to prove Theorem 1.4. While we shall not generally prove theorems in such a stylized form, it helps to think of proofs as very explicit lists of statements, each with a precise justification. In step (1), we have repeated one of the given statements of the theorem: that x is the sum of the squares of four integers. It often helps in proofs if we name quantities that are referred to but not named, and we have done so here, giving the four integers the names a, b, c, and d.

In step (2), we put down the other part of the hypothesis of the theorem: that the values being squared are each at least 1. Technically, this statement represents four distinct statements, one for each of the four integers involved.

	Statement	Justification
1.	$x = a^2 + b^2 + c^2 + d^2$	Given
2.	$a \geq 1$; $b \geq 1$; $c \geq 1$; $d \geq 1$	Given
3.	$a^2 \geq 1$; $b^2 \geq 1$; $c^2 \geq 1$; $d^2 \geq 1$	(2) and properties of arithmetic
4.	$x \geq 4$	(1), (3), and properties of arithmetic
5.	$2^x \geq x^2$	(4) and Theorem 1.3

Figure 1.3: A formal proof of Theorem 1.4

Then, in step (3) we observe that if a number is at least 1, then its square is also at least 1. We use as a justification the fact that statement (2) holds, and "properties of arithmetic." That is, we assume the reader knows, or can prove simple statements about how inequalities work, such as the statement "if $y \geq 1$, then $y^2 \geq 1$."

Step (4) uses statements (1) and (3). The first statement tells us that x is the sum of the four squares in question, and statement (3) tells us that each of the squares is at least 1. Again using well-known properties of arithmetic, we conclude that x is at least $1 + 1 + 1 + 1$, or 4.

At the final step (5), we use statement (4), which is the hypothesis of Theorem 1.3. The theorem itself is the justification for writing down its conclusion, since its hypothesis is a previous statement. Since the statement (5) that is the conclusion of Theorem 1.3 is also the conclusion of Theorem 1.4, we have now proved Theorem 1.4. That is, we have started with the hypothesis of that theorem, and have managed to deduce its conclusion. □

1.2.2 Reduction to Definitions

In the previous two theorems, the hypotheses used terms that should have been familiar: integers, addition, and multiplication, for instance. In many other theorems, including many from automata theory, the terms used in the statement may have implications that are less obvious. A useful way to proceed in many proofs is:

- If you are not sure how to start a proof, convert all terms in the hypothesis to their definitions.

Here is an example of a theorem that is simple to prove once we have expressed its statement in elementary terms. It uses the following two definitions:

1. A set S is *finite* if there exists an integer n such that S has exactly n elements. We write $\|S\| = n$, where $\|S\|$ is used to denote the number of elements in a set S. If the set S is not finite, we say S is *infinite*. Intuitively, an infinite set is a set that contains more than any integer number of elements.

2. If S and T are both subsets of some set U, then T is the *complement* of S (with respect to U) if $S \cup T = U$ and $S \cap T = \emptyset$. That is, each element of U is in exactly one of S and T; put another way, T consists of exactly those elements of U that are not in S.

Theorem 1.5: Let S be a finite subset of some infinite set U. Let T be the complement of S with respect to U. Then T is infinite.

PROOF: Intuitively, this theorem says that if you have an infinite supply of something (U), and you take a finite amount away (S), then you still have an infinite amount left. Let us begin by restating the facts of the theorem as in Fig. 1.4.

Original Statement	New Statement
S is finite	There is a integer n such that $\|S\| = n$
U is infinite	For no integer p is $\|U\| = p$
T is the complement of S	$S \cup T = U$ and $S \cap T = \emptyset$

Figure 1.4: Restating the givens of Theorem 1.5

We are still stuck, so we need to use a common proof technique called "proof by contradiction." In this proof method, to be discussed further in Section 1.3.3, we assume that the conclusion is false. We then use that assumption, together with parts of the hypothesis, to prove the opposite of one of the given statements of the hypothesis. We have then shown that it is impossible for all parts of the hypothesis to be true and for the conclusion to be false at the same time. The only possibility that remains is for the conclusion to be true whenever the hypothesis is true. That is, the theorem is true.

In the case of Theorem 1.5, the contradiction of the conclusion is "T is finite." Let us assume T is finite, along with the statement of the hypothesis that says S is finite; i.e., $\|S\| = n$ for some integer n. Similarly, we can restate the assumption that T is finite as $\|T\| = m$ for some integer m.

Now one of the given statements tells us that $S \cup T = U$, and $S \cap T = \emptyset$. That is, the elements of U are exactly the elements of S and T. Thus, there must be $n + m$ elements of U. Since $n + m$ is an integer, and we have shown $\|U\| = n + m$, it follows that U is finite. More precisely, we showed the number of elements in U is some integer, which is the definition of "finite." But the statement that U is finite contradicts the given statement that U is infinite. We have thus used the contradiction of our conclusion to prove the contradiction of one of the given statements of the hypothesis, and by the principle of "proof by contradiction" we may conclude the theorem is true. □

Proofs do not have to be so wordy. Having seen the ideas behind the proof, let us reprove the theorem in a few lines.